

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Лабораторная работа № 2

по дисциплине «Разработка защищённых Web-ресурсов»
на тему «Введение в объектно-ориентированное программирование. Классы
и объекты»

Выполнил обучающийся гр. ВКБ53
Ковалев Данил Петрович

Проверил: Ковальчик Р. В.

Ростов-на-Дону, 2026

1. Тема, цель и условие задания

Тема: введение в объектно-ориентированное программирование; классы, объекты, атрибуты и методы в Python.

Цель работы: научиться описывать состояние объектов в инициализаторе, изменять его методами, соблюдать ограничения предметной модели и разграничивать атрибуты класса и экземпляра.

По методическим указаниям необходимо дополнить два исходных файла. В `main.py` реализуется класс `BacteriaProducer`, управляющий колонией бактерий. В `task2.py` завершается класс `MushroomsCollector`, проверяющий и сохраняющий грибы в отдельных корзинах.

Требования к результату

Колония создаётся пустой и имеет заданную вместимость `max_bacteria`. Метод `create()` добавляет одну бактерию, `delete()` удаляет одну. Выход за границы от нуля до вместимости запрещён; успешные действия и отказы сопровождаются сообщениями из задания.

Каждая корзина должна иметь собственный список `mushrooms`. Метод `is_poisonous()` определяет принадлежность названия к двум запрещённым значениям из прекода; `add_mushroom()` добавляет допустимый гриб или печатает сообщение об отказе. Метод `__str__()` возвращает содержимое через запятую и пробел.

Основные понятия

Понятие	Применение в работе
Класс	Описание данных и действий: <code>BacteriaProducer</code> , <code>MushroomsCollector</code> .
Объект	Отдельная колония или корзина со своим состоянием.
Инициализатор <code>__init__</code>	Задаёт исходные атрибуты после создания экземпляра.
<code>self</code>	Ссылка на экземпляр, для которого вызван метод.
Атрибут экземпляра	<code>self.mushrooms</code> : отдельный список каждой корзины.
Атрибут класса	Значение на уровне класса; изменяемый список здесь был бы общим.
Метод объекта	Действие над конкретной колонией или корзиной.

2. Задание № 1. Реализация колонии бактерий

В инициализатор передаётся вместимость колонии. Она сохраняется в `self.max_bacteria`; счётчик `self.current_bacteria_count` получает значение 0. Оба атрибута принадлежат экземпляру, поэтому создание второй колонии не изменяет первую.

```
class BacteriaProducer:
    """Колония с ограничением численности бактерий."""

    def __init__(self, max_bacteria):
        self.max_bacteria = max_bacteria
        self.current_bacteria_count = 0

    def create(self):
        if self.current_bacteria_count >= self.max_bacteria:
            print('Нет места под новую бактерию')
        else:
            self.current_bacteria_count += 1
            print(
                'Добавлена одна бактерия. Бактерий в колонии: '
                f'{self.current_bacteria_count}'
            )

    def delete(self):
        if self.current_bacteria_count == 0:
            print('В популяции нет бактерий, удалять нечего')
        else:
            self.current_bacteria_count -= 1
            print(
                'Одна бактерия удалена. Бактерий в колонии: '
                f'{self.current_bacteria_count}'
            )
```

Листинг 1 — класс BacteriaProducer (main.py)

Метод `create()` сначала проверяет верхнюю границу. Условие `>=` включает требуемый случай равенства пределу. При отказе счётчик не меняется. В допустимом случае оператор `+=` увеличивает его ровно на единицу, затем выводится новое значение.

Метод `delete()` сначала проверяет, пуста ли колония. При нуле выводится отказ без изменения состояния. Иначе оператор `-=` уменьшает счётчик на единицу. В обоих методах `f`-строка подставляет текущее количество в сообщение.

Вместимость по смыслу задания — неотрицательное целое число. Валидация других типов и отрицательных значений в условии не задана и в учебный интерфейс не добавлена. Гарантия границ относится к работе через методы при такой вместимости и без внешнего изменения атрибутов.

3. Задание № 1. Запуск и анализ результата

Пример самопроверки из исходного файла сохранён. Он помещён под условие `if __name__ == "__main__"`, чтобы запускать демонстрацию при выполнении файла и не печатать её при импорте класса в тесты.

```
if __name__ == '__main__':
    # Пример запуска для самопроверки из задания.
    bacteria_producer = BacteriaProducer(max_bacteria=3)
    bacteria_producer.delete()
    bacteria_producer.create()
    bacteria_producer.create()
    bacteria_producer.create()
    bacteria_producer.create()
    bacteria_producer.create()
    bacteria_producer.delete()
```

Листинг 2 — демонстрационный сценарий main.py

```
$ python3 main.py
В популяции нет бактерий, удалять нечего
Добавлена одна бактерия. Бактерий в колонии: 1
Добавлена одна бактерия. Бактерий в колонии: 2
Добавлена одна бактерия. Бактерий в колонии: 3
Нет места под новую бактерию
Одна бактерия удалена. Бактерий в колонии: 2
```

Листинг 3 — фактический вывод первого задания

Шаг	Действие	До → после	Результат
1	delete()	0 → 0	Отказ: пустая колония
2	create()	0 → 1	Добавление
3	create()	1 → 2	Добавление
4	create()	2 → 3	Добавление
5	create()	3 → 3	Отказ: достигнут предел
6	delete()	3 → 2	Удаление

Почему сохраняются границы

Обозначим текущее количество через n , вместимость через M . Изначально $n = 0$. При $n < M$ добавление даёт $n + 1 \leq M$, а при $n \geq M$ изменения нет. При $n > 0$ удаление даёт $n - 1 \geq 0$, а при $n = 0$ изменения нет. Следовательно, после любой последовательности вызовов сохраняется $0 \leq n \leq M$.

4. Задание № 2. Реализация корзины грибов

В исходном классе список `mushrooms` был объявлен вне методов. Исправление состоит в создании `self.mushrooms = []` внутри `__init__`. Каждый вызов инициализатора создаёт новый список, связанный только с соответствующим объектом.

```
class MushroomsCollector:
    """Корзина с собственным списком грибов."""

    def __init__(self):
        self.mushrooms = []

    def is_poisonous(self, mushroom_name):
        return mushroom_name in ('Мухомор', 'Поганка')

    def add_mushroom(self, mushroom_name):
        if self.is_poisonous(mushroom_name):
            print('Нельзя добавить ядовитый гриб')
        else:
            self.mushrooms.append(mushroom_name)

    def __str__(self):
        return ', '.join(self.mushrooms)
```

Листинг 4 — класс MushroomsCollector (task2.py)

Проверка и добавление

Оператор `in` проверяет, совпадает ли `mushroom_name` с одним из элементов кортежа ("Мухомор", "Поганка"). Результат — логическое значение `True` или `False`. Метод не меняет содержимое корзины.

Метод `add_mushroom()` вызывает `is_poisonous()`. При `True` печатает точно заданное сообщение «Нельзя добавить ядовитый гриб». При `False` добавляет строку в конец списка методом `append()`. Порядок находок сохраняется; повторяющиеся названия допустимы, поскольку каждый найденный гриб учитывается отдельно.

Строковое представление

Метод `__str__()` возвращает строку, а не печатает её самостоятельно. Выражение `", ".join(self.mushrooms)` объединяет названия с разделителем «запятая и пробел». `print(collector)` использует это представление. Для пустой корзины возвращается пустая строка, для одного гриба — только его название.

Используется точное сравнение названий с учётом регистра, как в исходном примере. Нормализация ввода и полный справочник грибов заданием не предусмотрены. Это учебная проверка двух строк, а не определение съедобности реальных грибов.

5. Задание № 2. Ошибки прекода и проверка

Ошибка логического выражения

```
if mushroom_name == 'Мухомор' or 'Поганка':  
    return True
```

Листинг 5 — ошибочное условие из исходного файла

Выражение интерпретируется как `(mushroom_name == "Мухомор") or "Поганка"`. Непустая строка истинна в логическом контексте. Поэтому при несовпадении с «Мухомор» правая часть всё равно делает условие истинным, и любой гриб считался ядовитым. Исправленный оператор `in` явно сравнивает входное значение с обоими названиями.

Ошибка общего изменяемого состояния

При `mushrooms = []` в теле класса обращение `self.mushrooms` находило общий список класса. Вызов `append()` изменял этот список, поэтому находки одной корзины появлялись в остальных. Присваивание `self.mushrooms = []` в `__init__` создаёт отдельный атрибут экземпляра и отдельный список.

```
if __name__ == '__main__':  
    # Пример запуска для самопроверки из задания.  
    collector_1 = MushroomsCollector()  
    collector_1.add_mushroom('Мухомор')  
    collector_1.add_mushroom('Подосиновик')  
    collector_1.add_mushroom('Белый')  
    print(collector_1)  
  
    collector_2 = MushroomsCollector()  
    collector_2.add_mushroom('Лисичка')  
    print(collector_1)  
    print(collector_2)
```

Листинг 6 — демонстрационный сценарий task2.py

```
$ python3 task2.py  
Нельзя добавить ядовитый гриб  
Подосиновик, Белый  
Подосиновик, Белый  
Лисичка
```

Листинг 7 — фактический вывод второго задания

«Мухомор» отклонён. Первая корзина содержит «Подосиновик, Белый». После добавления «Лисичка» во вторую корзину повторный вывод первой остаётся прежним. Это демонстрирует независимость объектов; в тесте также проверено, что их списки не являются одним объектом.

6. Окружение, структура и воспроизведение

Решение выполнено на Python 3.9.6. Для программ и тестов достаточно стандартной библиотеки Python; установка сторонних пакетов и запуск веб-сервера не требуются. Пути ниже указаны относительно каталога лабораторной работы № 2.

```
main.py           # колония бактерий
task2.py          # корзина грибов
test_lab2.py      # автоматические проверки
README.md         # инструкция запуска
docs/verification.txt # фактический протокол
docs/build_report.py # сборка отчёта DOCX
docs/Ковалев Д.П. ВКБ53 ЛР2.docx
```

Листинг 8 — структура подготовленных материалов

Порядок запуска

Открыть терминал в каталоге с `main.py` и `task2.py`. Последовательно выполнить приведённые команды. Первые две запускают исходные демонстрации; третья выполняет тесты и выводит результаты каждого сценария.

```
python3 main.py
python3 task2.py
python3 -m unittest -v test_lab2
```

Листинг 9 — команды воспроизведения

На системе, где интерпретатор доступен под именем `python`, используется это имя вместо `python3`. Программы не запрашивают ввод: параметры и последовательность действий заданы в демонстрационных блоках. Для другого сценария можно импортировать класс и вызвать его методы.

```
from main import BacteriaProducer
from task2 import MushroomsCollector

colony = BacteriaProducer(max_bacteria=1)
colony.create()
colony.delete()

basket = MushroomsCollector()
basket.add_mushroom("Белый")
print(basket)
```

Листинг 10 — пример использования классов из другого модуля

Файл `docs/verification.txt` содержит вывод реально выполненных команд, версию Python и коды завершения. Сборка DOCX вынесена в отдельный вспомогательный скрипт и использует `python-docx` и образец отчёта ЛР1; эти зависимости не нужны для выполнения заданий.

7. Автоматическая проверка результата

Проверки реализованы в `test_lab2.py` на `unittest`. Для сравнения сообщений `redirect_stdout` направляет вывод в `StringIO`. Проверяется не только состояние объектов, но и точный текст, включая завершающий перевод строки. Все 12 тестов завершились успешно.

Тест / группа	Проверяемое поведение
Начальное состояние	Вместимость сохраняется; счётчик равен нулю.
Добавление и предел	Увеличение на 1; отказ при заполнении; точные сообщения.
Удаление и пустая колония	Уменьшение на 1; отказ при нуле; точные сообщения.
Нулевая вместимость	Добавление запрещено, счётчик остаётся нулевым.
Независимость колоний	Изменение одной колонии не влияет на другую.
Последовательности операций	Все 256 последовательностей длины 8 для $M = 0, 1, 2, 3$.
Пустая корзина	Пустой список и пустое строковое представление.
Два ядовитых названия	True для обоих; отказ без добавления.
Допустимые названия	False; добавление без сообщений; порядок и разделитель.
Независимость корзин	Разные списки; содержимое каждой сохраняется отдельно.
Повторяющиеся грибы	Оба экземпляра одного названия учитываются.
Отказ после добавления	Ядовитый гриб не меняет уже заполненную корзину.

В переборной проверке выполнено $4 \times 256 = 1024$ сценария, всего 8192 операции. После каждого шага фактический счётчик сравнивается с эталоном: $\min(M, n + 1)$ для добавления и $\max(0, n - 1)$ для удаления. Это проверяет возврат к допустимым операциям после отказа, многократные отказы и чередование действий.

Ran 12 tests in 0.011s

ОК

Код завершения: 0

Листинг 11 — итог фактического запуска unittest

Демонстрационные программы также завершились с кодом 0. Требования задания выполнены: границы численности соблюдены, ошибки вывода и общего списка устранены, строковое представление корзины соответствует образцу.

8. Ответы по теории, вывод и источники

В PDF отдельный перечень контрольных вопросов не приведён. Ниже раскрыты понятия из раздела «Что необходимо знать для выполнения работы».

Чем класс отличается от объекта?

Класс описывает набор атрибутов и методов. Объект — конкретный экземпляр этого описания. Один класс `MushroomsCollector` позволяет создать несколько корзин, каждая со своим содержимым.

Для чего нужен инициализатор?

Метод `__init__` автоматически вызывается для инициализации экземпляра. Он сохраняет переданные параметры и задаёт начальное состояние: вместимость, нулевой счётчик или пустой список.

Как связаны метод объекта и `self`?

Метод работает с конкретным экземпляром. При вызове `colony.create()` Python передаёт `colony` первым аргументом `self`. Поэтому `self.current_bacteria_count` относится к счётчику именно этой колонии.

Чем отличаются атрибуты класса и объекта?

Атрибут класса определяется в теле класса и доступен экземплярам через поиск атрибутов. Атрибут экземпляра создаётся присваиванием `self.name`. Общий изменяемый список неудобен для независимых корзин: его изменение видно всем объектам, использующим этот список.

Вывод

Выполнены оба задания лабораторной работы. Реализован класс `BacteriaProducer` с ограничением численности и сообщениями об операциях. Исправлен и дополнен `MushroomsCollector`: каждая корзина хранит собственные данные, ядовитые названия отклоняются, содержимое выводится через запятую и пробел. Освоены инициализация, методы экземпляра, логические проверки и метод `__str__`. Работоспособность подтверждена исходными примерами и 12 автоматическими тестами, включая граничные состояния и независимость объектов.

Использованные материалы

1. ДГТУ. Лабораторная работа № 2 «Введение в объектно-ориентированное программирование. Классы и объекты». Предоставленный файл «Лабораторная_работа_2.pdf», с. 1–3.
2. Исходные учебные файлы `main.py` и `task2.py`, предоставленные вместе с заданием.
3. Ковалев Д. П. Отчёт по лабораторной работе № 1, группа ВКБ53. Использован как образец титульного листа и оформления.